Assignment 1(Chapter 1-2)

Part B – Lab Questions

Name: Pramathesh Kumar Sahoo

redg No: 2341019043

sec : 23412N3 Serial: 52

In [1]:
```
import numpy as np
import pandas as pd
```

Q1

In [2]:
```
arr = np.arange(1, 21)

def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True
primes = arr[[is_prime(x) for x in arr]]

mean = np.mean(primes)
variance = np.var(primes)

print("Array:", arr)
print("Primes:", primes)
print("Mean:", mean)
print("Variance:", variance)
```

```
Array: [ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20]
Primes: [ 2  3  5  7 11 13 17 19]
Mean: 9.625
Variance: 35.734375
```

Q2

In [3]:
```
arr = np.arange(1, 17).reshape(4, 4)
sub_matrix = arr[2:, :2]
determinant = np.linalg.det(sub_matrix)
print(arr)
print(sub_matrix)
print(determinant)
```

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]
 [13 14 15 16]]
[[ 9 10]
 [13 14]]
-4.0000000000000036
```

Q3

In [4]:
```
import pandas as pd

data = {
    'Name': ['A', 'B', 'C', 'D', 'E'],
    'Math': [85, 92, 78, 90, 88],
    'Science': [80, 95, 82, 89, 84],
    'English': [78, 88, 85, 92, 90]
}

df = pd.DataFrame(data)
df['Total'] = df[['Math', 'Science', 'English']].sum(axis=1)
df['Average'] = df[['Math', 'Science', 'English']].mean(axis=1)
topper = df.loc[df['Average'].idxmax()]
print(df)
print("Topper:", topper['Name'], "with average:", topper['Average'])
```

```
  Name  Math  Science  English  Total    Average
0    A    85       80       78    243  81.000000
1    B    92       95       88    275  91.666667
2    C    78       82       85    245  81.666667
3    D    90       89       92    271  90.333333
4    E    88       84       90    262  87.333333
Topper: B with average: 91.66666666666667
```

Q4

In [5]:
```
arr = np.random.randint(0, 2, 1000)
heads = np.sum(arr == 1)
tails = np.sum(arr == 0)
prob_heads = heads / 1000
#print(arr)
print("Heads:", heads, "Tails:", tails)
print("Probability of heads:", prob_heads)
```

```
Heads: 508 Tails: 492
Probability of heads: 0.508
```

Q5

In [6]:
```
data = {'ID':[1,2,3,4,5],'Name':['A','B','C','D','E'],'Salary':[50000,60000,55000,6
df = pd.DataFrame(data)
df['Bonus'] = df['Salary']*0.1
above_avg = df[df['Salary'] > df['Salary'].mean()]
```

```
print(df)
print(above_avg)
```

```
   ID Name  Salary   Bonus
0   1    A   50000  5000.0
1   2    B   60000  6000.0
2   3    C   55000  5500.0
3   4    D   65000  6500.0
4   5    E   70000  7000.0
   ID Name  Salary   Bonus
3   4    D   65000  6500.0
4   5    E   70000  7000.0
```

Q6

In [7]:
```
arr = np.arange(1,10).reshape(3,3)
transpose = arr.T
inverse = np.linalg.inv(arr)
identity_approx = np.dot(arr, inverse)
print(arr)
print(transpose)
print(inverse)
print(identity_approx)
```

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
[[1 4 7]
 [2 5 8]
 [3 6 9]]
[[ 3.15251974e+15 -6.30503948e+15  3.15251974e+15]
 [-6.30503948e+15  1.26100790e+16 -6.30503948e+15]
 [ 3.15251974e+15 -6.30503948e+15  3.15251974e+15]]
[[ 0.   1.  -0.5]
 [ 0.   2.  -1. ]
 [ 0.   3.   2.5]]
```

Q7

In [8]:
```
temps = np.random.randint(20, 41, 30)
hottest = temps.max()
coldest = temps.min()
mean_temp = temps.mean()
median_temp = np.median(temps)
std_temp = temps.std()
print(temps)
print("Hottest:", hottest, "Coldest:", coldest)
print("Mean:", mean_temp, "Median:", median_temp, "Std:", std_temp)
```

```
[31 30 22 21 39 37 21 36 23 28 21 37 34 30 33 25 20 28 34 32 40 29 34 30
 26 28 36 33 35 23]
Hottest: 40 Coldest: 20
Mean: 29.866666666666667 Median: 30.0 Std: 5.789262090763861
```

Q8

In [9]:
```
marks = pd.Series([35, 42, 55, 28, 90, 61, 39, 47])
marks_replaced = marks.apply(lambda x: 'Fail' if x<40 else x)
passed_count = (marks_replaced != 'Fail').sum()
print(marks_replaced)
print("Passed:", passed_count)
```

```
0    Fail
1      42
2      55
3    Fail
4      90
5      61
6    Fail
7      47
dtype: object
Passed: 5
```

Q9

In [10]:
```
dice = np.random.randint(1,7,500)
counts = np.bincount(dice)[1:]  # faces 1-6
rel_freq = counts/500
print(counts)
print(rel_freq)
```

```
[78 68 92 76 91 95]
[0.156 0.136 0.184 0.152 0.182 0.19 ]
```

Q10

In [11]:
```
data = {'Name':['P1','P2','P3','P4','P5','P6'],'Quantity':[10,5,8,12,7,6],'Price':[
df = pd.DataFrame(data)
df['Total'] = df['Quantity']*df['Price']
max_sales = df.loc[df['Total'].idxmax()]
print(df)
print("Max sales product:", max_sales['Name'], "Revenue:", max_sales['Total'])
```

```
  Name  Quantity  Price  Total
0   P1        10    100   1000
1   P2         5    200   1000
2   P3         8    150   1200
3   P4        12     80    960
4   P5         7    120    840
5   P6         6    250   1500
Max sales product: P6 Revenue: 1500
```

Q11

In [12]:
```
data = {'A':[1,2,np.nan,4],'B':[np.nan,2,3,np.nan],'C':[1,np.nan,3,4]}
df = pd.DataFrame(data)
df_filled = df.fillna(df.mean())
df_dropped = df.dropna(thresh=df.shape[1]-1)
print(df)
```

```
print(df_filled)
print(df_dropped)
```

```
     A    B    C
0  1.0  NaN  1.0
1  2.0  2.0  NaN
2  NaN  3.0  3.0
3  4.0  NaN  4.0
          A    B         C
0  1.000000  2.5  1.000000
1  2.000000  2.0  2.666667
2  2.333333  3.0  3.000000
3  4.000000  2.5  4.000000
     A    B    C
0  1.0  NaN  1.0
1  2.0  2.0  NaN
2  NaN  3.0  3.0
3  4.0  NaN  4.0
```

Q12

In [13]:
```
arr = np.arange(1,31)
matrix = arr.reshape(5,6)
even_numbers = matrix[matrix % 2 == 0]
avg_even = even_numbers.mean()
print(matrix)
print(even_numbers)
print(avg_even)
```

```
[[ 1  2  3  4  5  6]
 [ 7  8  9 10 11 12]
 [13 14 15 16 17 18]
 [19 20 21 22 23 24]
 [25 26 27 28 29 30]]
[ 2  4  6  8 10 12 14 16 18 20 22 24 26 28 30]
16.0
```

Q13

In [14]:
```
data = {'Name':['A','B','C','D','E','F'],'Age':[19,21,18,22,20,17],'Marks':[75,60,8
df = pd.DataFrame(data)
above_avg = df[df['Marks'] > df['Marks'].mean()]
young_high = df[(df['Age']<20) & (df['Marks']>60)]
print(df)
print(above_avg)
print(young_high['Name'])
```

```
  Name  Age  Marks
0    A   19     75
1    B   21     60
2    C   18     80
3    D   22     55
4    E   20     90
5    F   17     65
  Name  Age  Marks
0    A   19     75
2    C   18     80
4    E   20     90
0    A
2    C
5    F
Name: Name, dtype: object
```

Q14

In [15]:
```
data = {'Name':['A','B','C','D','E'],'Tasks_Completed':[50,60,45,70,55],'Hours_Work
df = pd.DataFrame(data)
df['Efficiency'] = df['Tasks_Completed']/df['Hours_Worked']
top_employee = df.loc[df['Efficiency'].idxmax()]
print(df)
print("Top employee:", top_employee['Name'], "Efficiency:", top_employee['Efficienc
```

```
  Name  Tasks_Completed  Hours_Worked  Efficiency
0    A               50            10         5.0
1    B               60            12         5.0
2    C               45             9         5.0
3    D               70            14         5.0
4    E               55            11         5.0
Top employee: A Efficiency: 5.0
```